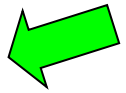
Frequent Patterns

Big Data Analytics

Readings

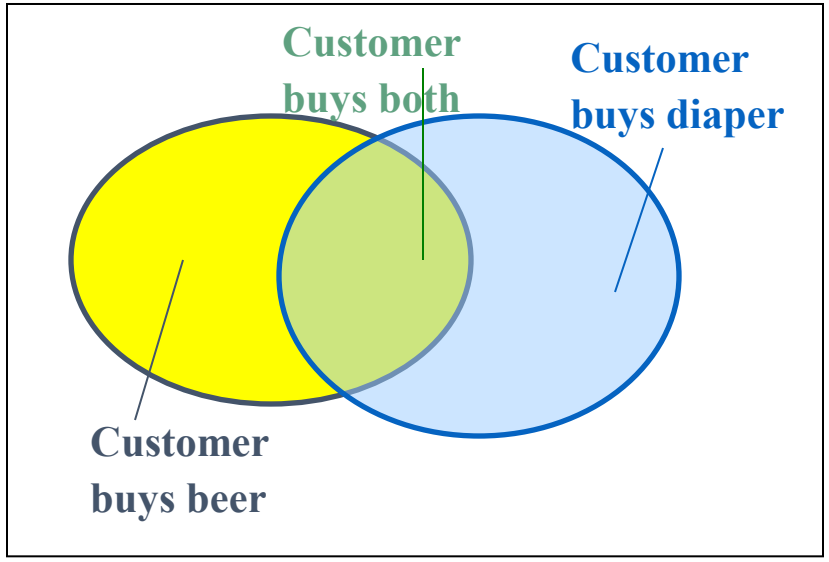
- “Data Mining – Concepts and Techniques” by Jiawei Han, Micheline Kamber, Jian Pei, third edition
  - Chap. 6

Chapter 6: Mining Frequent Patterns, Association and Correlations: Basic Concepts and Methods

- Basic Concepts 
- Frequent Itemset Mining Methods
- Which Patterns Are Interesting?—Pattern Evaluation Methods
- Summary

Basic Concepts: Frequent Patterns

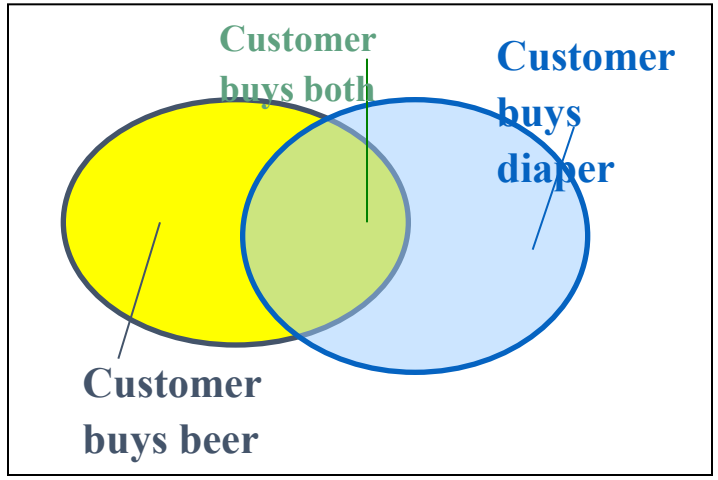
Tid	Items bought
10	Beer, Nuts, Diaper
20	Beer, Coffee, Diaper
30	Beer, Diaper, Eggs
40	Nuts, Eggs, Milk
50	Nuts, Coffee, Diaper, Eggs, Milk



- itemset: A set of one or more items
- k-itemset $X = \{x_1, \ldots, x_k\}$
- *(absolute) support*, or, *support count* of X: Frequency or occurrence of an itemset X
- *(relative) support*, *s*, is the fraction of transactions that contains X (i.e., the probability that a transaction contains X)
- An itemset X is *frequent* if X's support is no less than a *minsup* threshold

Basic Concepts: Association Rules

Tid	Items bought
10	Beer, Nuts, Diaper
20	Beer, Coffee, Diaper
30	Beer, Diaper, Eggs
40	Nuts, Eggs, Milk
50	Nuts, Coffee, Diaper, Eggs, Milk



- Find all the rules $X \rightarrow Y$ with minimum support and confidence
  - support, *s*, probability that a transaction contains $X \cup Y$
  - confidence, *c*, conditional probability that a transaction having X also contains *Y*

Let minsup = 50%, minconf = 50%

Freq. Pat.: Beer:3, Nuts:3, Diaper:4, Eggs:3, {Beer, Diaper}:3

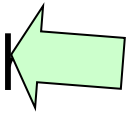
- Association rules: (many more!)
  - *Beer* → *Diaper* (60%, 100%)
  - *Diaper* → *Beer* (60%, 75%)

Computational Complexity of Frequent Itemset Mining

- How many itemsets are potentially to be generated in the worst case?
  - The number of frequent itemsets to be generated is senstive to the minsup threshold
  - When minsup is low, there exist potentially an exponential number of frequent itemsets

Tid	Items bought
10	Beer, Nuts, Diaper
20	Beer, Coffee, Diaper
30	Beer, Diaper, Eggs
40	Nuts, Eggs, Milk
50	Nuts, Coffee, Diaper, Eggs, Milk

Scalable Frequent Itemset Mining Methods

- Apriori: A Candidate Generation-and-Test Approach 
- Improving the Efficiency of Apriori
- FPGrowth: A Frequent Pattern-Growth Approach
- ECLAT: Frequent Pattern Mining with Vertical Data Format

The Downward Closure Property and Scalable Mining Methods - Apriori

- The downward closure property of frequent patterns
  - Any subset of a frequent itemset must be frequent
  - If **{beer, diaper, nuts}** is frequent, so is **{beer, diaper}**
  - i.e., every transaction having {beer, diaper, nuts} also contains {beer, diaper}

TID	Items
1	Beer, Diaper, Eggs
2	Beer, Diaper, Nuts
3	Diaper, Nuts, Eggs, Coffee
4	Beer, Diaper, Nuts, Eggs, Coffee, Milk

Apriori: A Candidate Generation & Test Approach

- Apriori pruning principle: If there is any itemset which is infrequent, its superset should not be generated/tested! (Agrawal & Srikant @VLDB'94, Mannila, et al. @ KDD' 94)
- Method:
  - Initially, scan DB once to get frequent 1-itemset
    - {Beer}, {Nuts}, {Diaper}, {Eggs}

Apriori: A Candidate Generation & Test Approach

- Apriori pruning principle: If there is any itemset which is infrequent, its superset should not be generated/tested! (Agrawal & Srikant @VLDB'94, Mannila, et al. @ KDD' 94)
- Method:
  - Initially, scan DB once to get frequent 1-itemset
    - {Beer}, {Nuts}, {Diaper}, {Eggs}
  - Generate length (k+1) candidate itemsets from length k frequent itemsets

Apriori: A Candidate Generation & Test Approach

- Apriori pruning principle: If there is any itemset which is infrequent, its superset should not be generated/tested! (Agrawal & Srikant @VLDB'94, Mannila, et al. @ KDD' 94)
- Method:
  - Initially, scan DB once to get frequent 1-itemset
    - {Nuts}, {Diaper}, {Eggs}
  - Generate length (k+1) candidate itemsets from length k frequent itemsets
    - {Beer, Nuts}, {Beer, Diaper}, {Beer, Eggs}

Apriori: A Candidate Generation & Test Approach

- Apriori pruning principle: If there is any itemset which is infrequent, its superset should not be generated/tested! (Agrawal & Srikant @VLDB'94, Mannila, et al. @ KDD' 94)
- Method:
  - Initially, scan DB once to get frequent 1-itemset
    - {Diaper}, {Eggs}
  - Generate length (k+1) candidate itemsets from length k frequent itemsets
    - {Beer, Nuts}, {Beer, Diaper}, {Beer, Eggs}, {Nuts, Diaper}, {Nuts, Eggs}

Apriori: A Candidate Generation & Test Approach

- Apriori pruning principle: If there is any itemset which is infrequent, its superset should not be generated/tested! (Agrawal & Srikant @VLDB'94, Mannila, et al. @ KDD' 94)
- Method:
  - Initially, scan DB once to get frequent 1-itemset
    - {Eggs}
  - Generate length (k+1) candidate itemsets from length k frequent itemsets
    - {Beer, Nuts}, {Beer, Diaper}, {Beer, Eggs}, {Nuts, Diaper}, {Nuts, Eggs}, {Diaper, Eggs}

Apriori: A Candidate Generation & Test Approach

- Apriori pruning principle: If there is any itemset which is infrequent, its superset should not be generated/tested! (Agrawal & Srikant @VLDB'94, Mannila, et al. @ KDD' 94)
- Method:
  - Initially, scan DB once to get frequent 1-itemset
    - {Beer}, {Nuts}, {Diaper}, {Eggs}
  - Generate length (k+1) candidate itemsets from length k frequent itemsets
    - {Beer, Nuts}, {Beer, Diaper}, {Beer, Eggs}, {Nuts, Diaper}, {Nuts, Eggs}, {Diaper, Eggs}

Apriori: A Candidate Generation & Test Approach

- Apriori pruning principle: If there is any itemset which is infrequent, its superset should not be generated/tested! (Agrawal & Srikant @VLDB'94, Mannila, et al. @ KDD' 94)
- Method:
  - Initially, scan DB once to get frequent 1-itemset
    - {Beer}, {Nuts}, {Diaper}, {Eggs}
  - Generate length (k+1) candidate itemsets from length k frequent itemsets
    - {Beer, Nuts}, {Beer, Diaper}, {Beer, Eggs}, {Nuts, Diaper}, {Nuts, Eggs}, {Diaper, Eggs}
  - Test the candidates against DB

Tid	Items bought
10	Beer, Nuts, Diaper
20	Beer, Coffee, Diaper
30	Beer, Diaper, Eggs
40	Nuts, Eggs, Milk
50	Nuts, Coffee, Diaper, Eggs, Milk

Apriori: A Candidate Generation & Test Approach

- Apriori pruning principle: If there is any itemset which is infrequent, its superset should not be generated/tested! (Agrawal & Srikant @VLDB'94, Mannila, et al. @ KDD' 94)
- Method:
  - Initially, scan DB once to get frequent 1-itemset
    - {Beer}, {Nuts}, {Diaper}, {Eggs}
  - Generate length (k+1) candidate itemsets from length k frequent itemsets
    - {Beer, Nuts}, {Beer, Diaper}, {Beer, Eggs}, {Nuts, Diaper}, {Nuts, Eggs}, {Diaper, Eggs}
  - Test the candidates against DB
  - Terminate when no frequent or candidate set can be generated

Implementation of Apriori

- How to generate candidates?
  - Step 1: self-joining L_k
  - Step 2: pruning
- Example of Candidate-generation
  - L_3={*abc, abd, acd, ace, bcd*}
  - Self-joining: L_3*L_3
    - *abcd* from *abc* and *abd*
    - *acde* from *acd* and *ace*
  - Pruning:
    - *acde* is removed because *ade* is not in L_3
  - C_4 = {*abcd*}

The Apriori Algorithm—An Example with Sup_{min} = 2

Database TDB

Tid	Items
10	A, C, D
20	B, C, E
30	A, B, C, E
40	B, E

C_1

1st scan →

The Apriori Algorithm—An Example with Sup_{min} = 2

Database TDB

Tid	Items
10	A, C, D
20	B, C, E
30	A, B, C, E
40	B, E

C_1

1st scan

Itemset	sup
{A}	2
{B}	3
{C}	3
{D}	1
{E}	3

L_1

The Apriori Algorithm—An Example with Sup_{min} = 2

Database TDB

Tid	Items
10	A, C, D
20	B, C, E
30	A, B, C, E
40	B, E

C_1

1st scan

Itemset	sup
{A}	2
{B}	3
{C}	3
{D}	1
{E}	3

L_1

Itemset	sup
{A}	2
{B}	3
{C}	3
{E}	3

The Apriori Algorithm—An Example with Sup_{min} = 2

Database TDB

Tid	Items
10	A, C, D
20	B, C, E
30	A, B, C, E
40	B, E

C_1

1st scan

Itemset	sup
{A}	2
{B}	3
{C}	3
{D}	1
{E}	3

L_1

Itemset	sup
{A}	2
{B}	3
{C}	3
{E}	3

C_2

Itemset
{A, B}
{A, C}
{A, E}
{B, C}
{B, E}
{C, E}

The Apriori Algorithm—An Example with Sup_{min} = 2

Database TDB

Tid	Items
10	A, C, D
20	B, C, E
30	A, B, C, E
40	B, E

1st scan → C_1

Itemset	sup
{A}	2
{B}	3
{C}	3
{D}	1
{E}	3

L_1 →

Itemset	sup
{A}	2
{B}	3
{C}	3
{E}	3

C_2 ← 2nd scan

Itemset
{A, B}
{A, C}
{A, E}
{B, C}
{B, E}
{C, E}

The Apriori Algorithm—An Example with Sup_{min} = 2

Database TDB

Tid	Items
10	A, C, D
20	B, C, E
30	A, B, C, E
40	B, E

1st scan

C_1

Itemset	sup
{A}	2
{B}	3
{C}	3
{D}	1
{E}	3

L_1

Itemset	sup
{A}	2
{B}	3
{C}	3
{E}	3

C_2

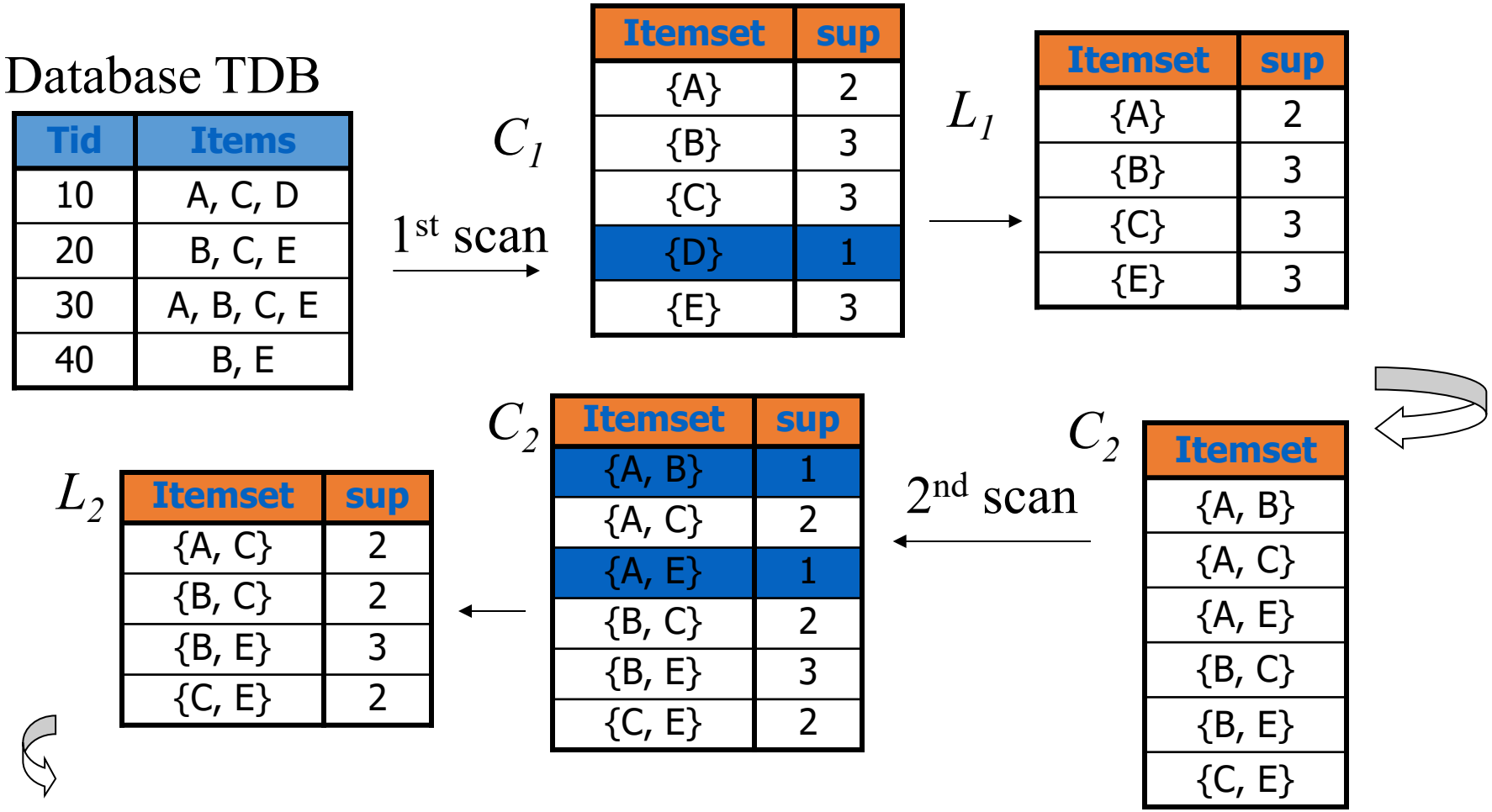
Itemset	sup
{A, B}	1
{A, C}	2
{A, E}	1
{B, C}	2
{B, E}	3
{C, E}	2

2nd scan

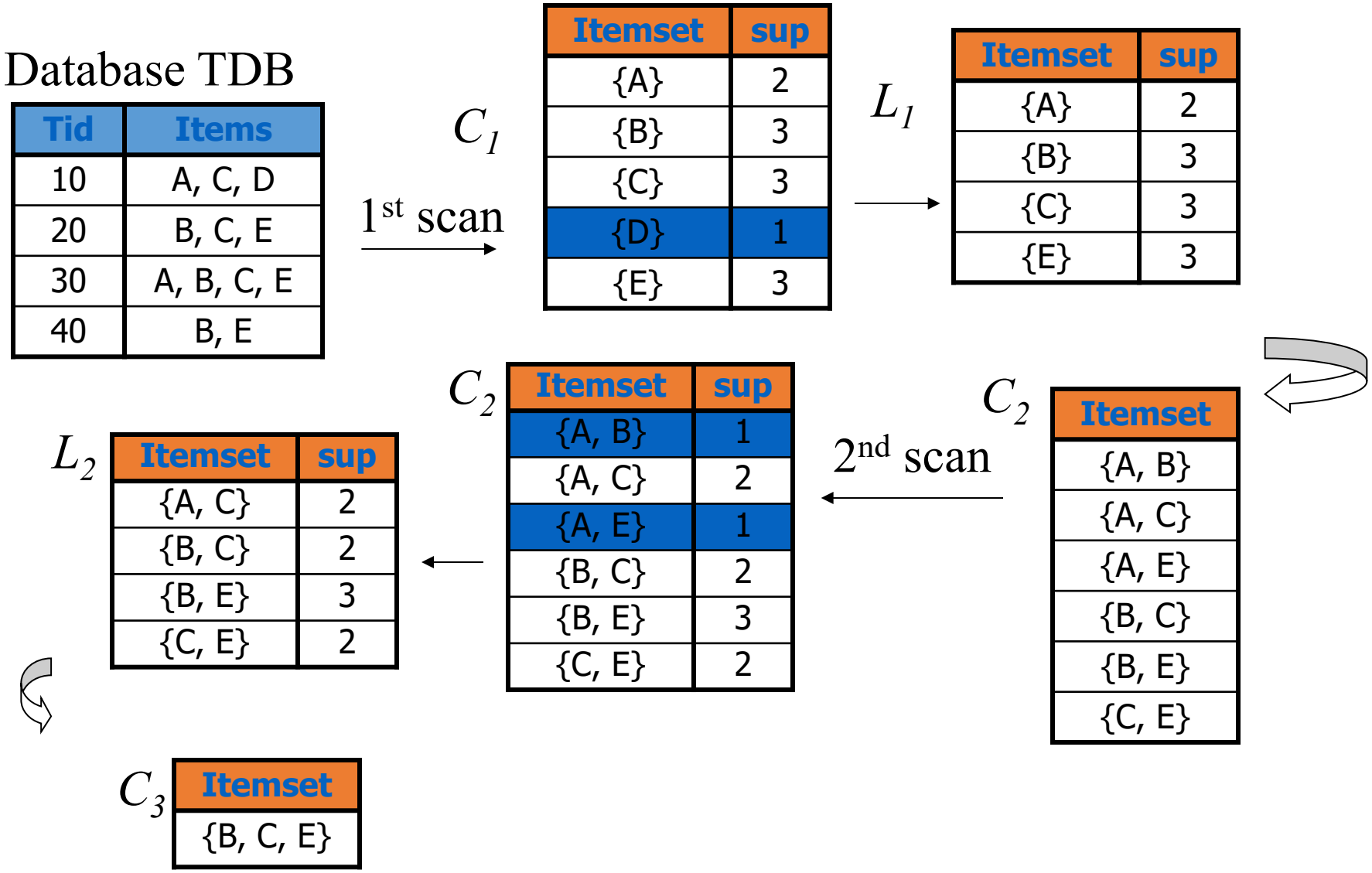
C_2

Itemset
{A, B}
{A, C}
{A, E}
{B, C}
{B, E}
{C, E}

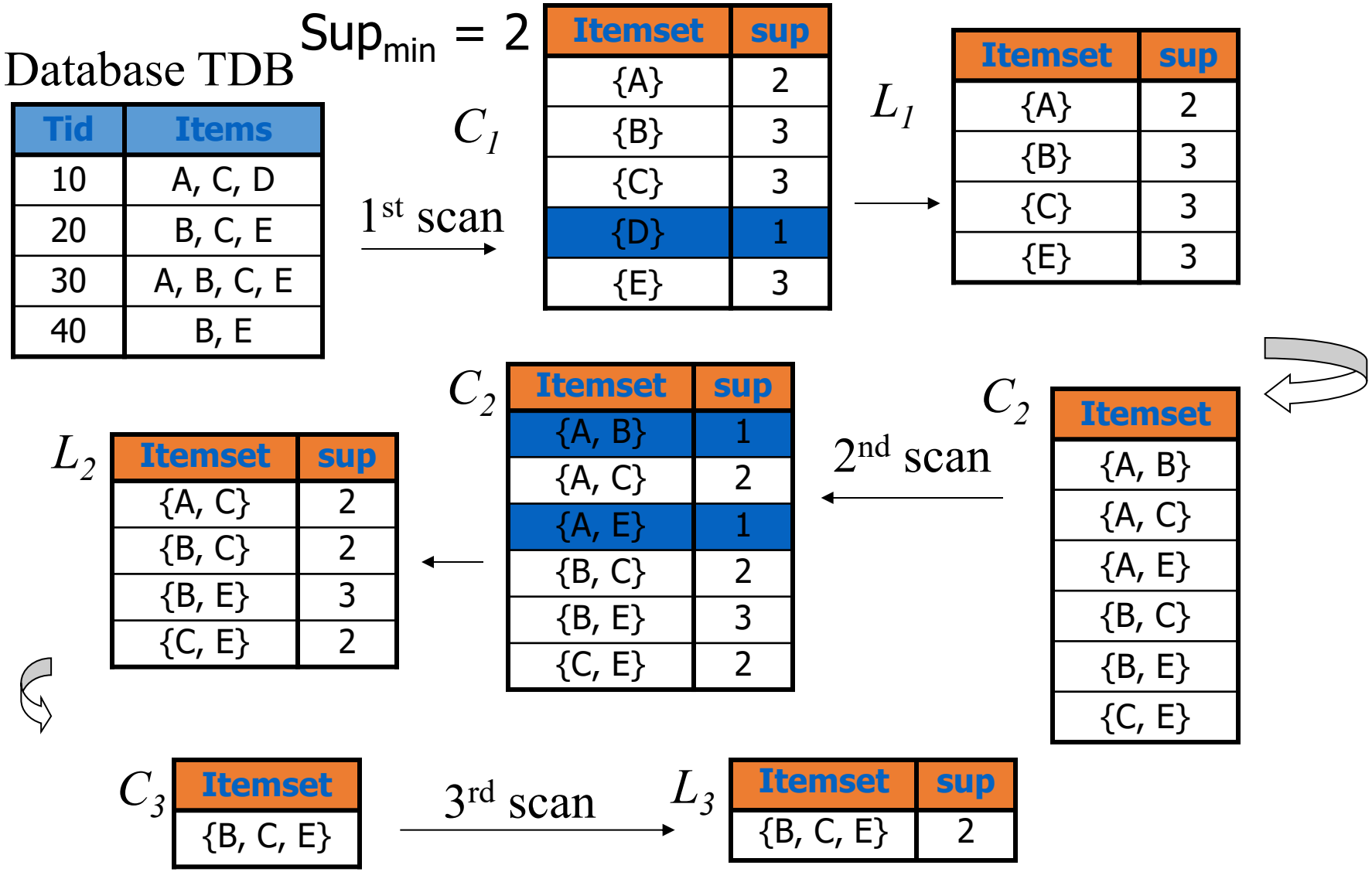
The Apriori Algorithm—An Example with Sup_{min} = 2



The Apriori Algorithm—An Example with Sup_{min} = 2



The Apriori Algorithm—An Example



The Apriori Algorithm—Another Example with Sup_{min} = 2

Transactional Data for an *AllElectronics* Branch

TID	*List of item_IDs*
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

The Apriori Algorithm—Another Example with Sup_{min} = 2

Transactional Data for an *AllElectronics* Branch

TID	*List of item_IDs*
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

Scan D for count of each candidate →

C_1

Itemset	Sup. count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

The Apriori Algorithm—Another Example with Sup_{min} = 2

Transactional Data for an *AllElectronics* Branch

TID	*List of item_IDs*
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

Scan D for count of each candidate →

C_1

Itemset	Sup. count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

Compare candidate support count with minimum support count →

L_1

Itemset	Sup. count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

The Apriori Algorithm—Another Example with Sup_{min} = 2

Transactional Data for an *AllElectronics* Branch

TID	*List of item_IDs*
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

Scan D for count of each candidate →

C_1

Itemset	Sup. count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

Compare candidate support count with minimum support count →

L_1

Itemset	Sup. count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

Generate C_2 candidates from L_1 →

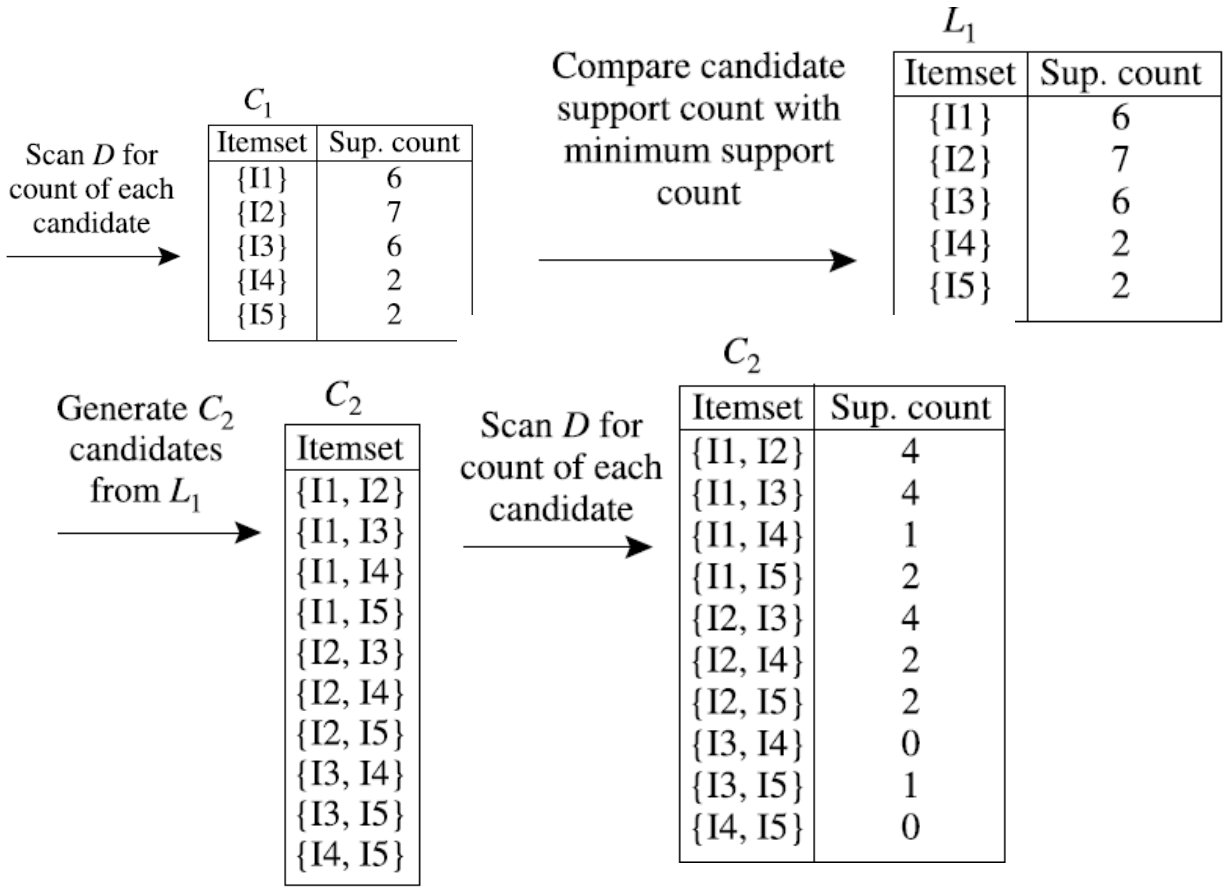
C_2

Itemset
{I1, I2}
{I1, I3}
{I1, I4}
{I1, I5}
{I2, I3}
{I2, I4}
{I2, I5}
{I3, I4}
{I3, I5}
{I4, I5}

The Apriori Algorithm—Another Example with Sup_{min} = 2

Transactional Data for an *AllElectronics* Branch

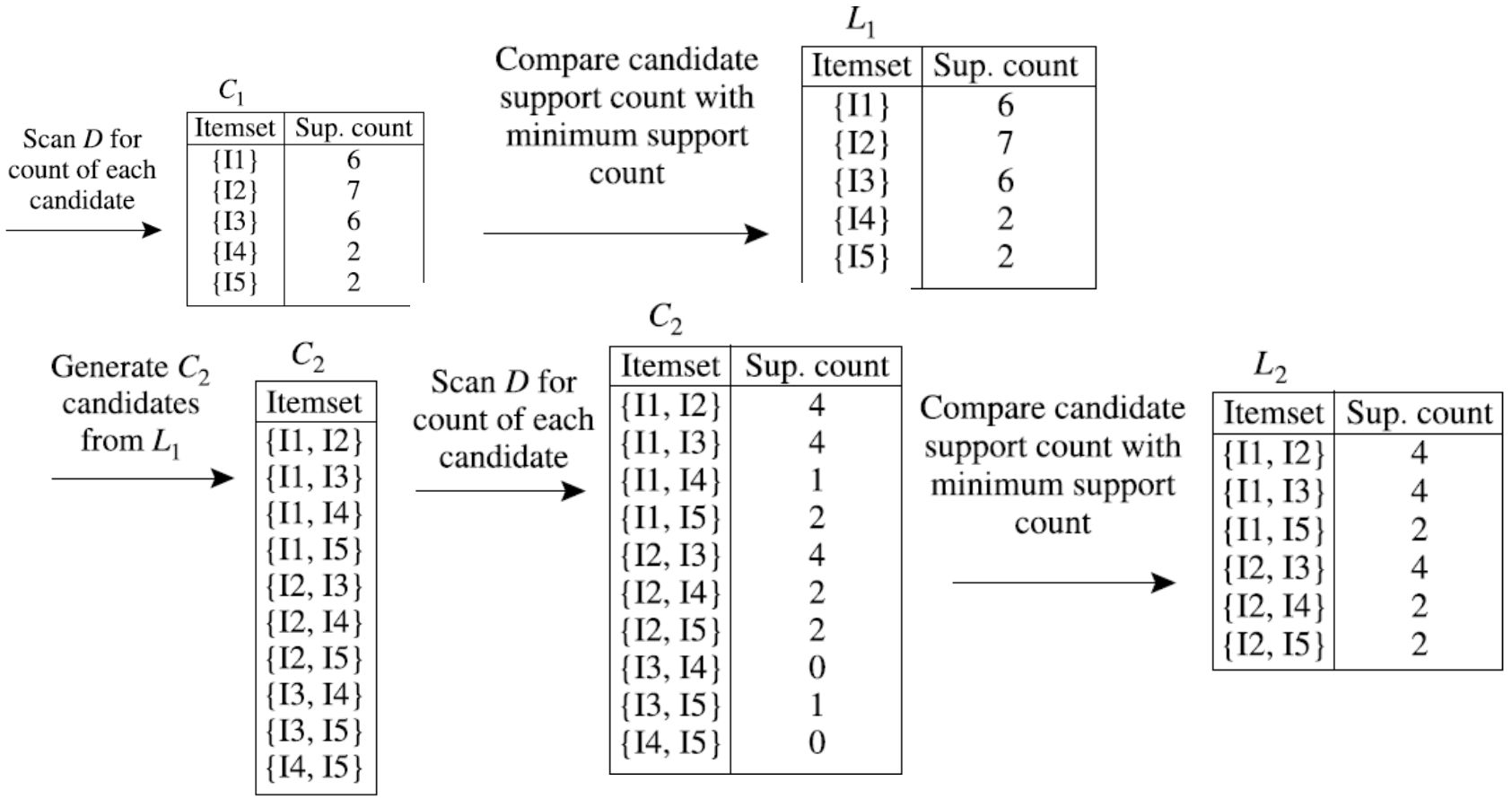
TID	*List of item_IDs*
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3



The Apriori Algorithm—Another Example with Sup_{min} = 2

Transactional Data for an *AllElectronics* Branch

TID	*List of item_IDs*
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3



The Apriori Algorithm—Another Example with Sup_{min} = 2

Transactional Data for an *AllElectronics* Branch

TID	*List of item_IDs*
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

Scan D for count of each candidate →

C_1

Itemset	Sup. count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

Compare candidate support count with minimum support count →

L_1

Itemset	Sup. count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

Generate C_2 candidates from L_1 →

C_2

Itemset
{I1, I2}
{I1, I3}
{I1, I4}
{I1, I5}
{I2, I3}
{I2, I4}
{I2, I5}
{I3, I4}
{I3, I5}
{I4, I5}

Scan D for count of each candidate →

C_2

Itemset	Sup. count
{I1, I2}	4
{I1, I3}	4
{I1, I4}	1
{I1, I5}	2
{I2, I3}	4
{I2, I4}	2
{I2, I5}	2
{I3, I4}	0
{I3, I5}	1
{I4, I5}	0

Compare candidate support count with minimum support count →

L_2

Itemset	Sup. count
{I1, I2}	4
{I1, I3}	4
{I1, I5}	2
{I2, I3}	4
{I2, I4}	2
{I2, I5}	2

Generate C_3 candidates from L_2 →

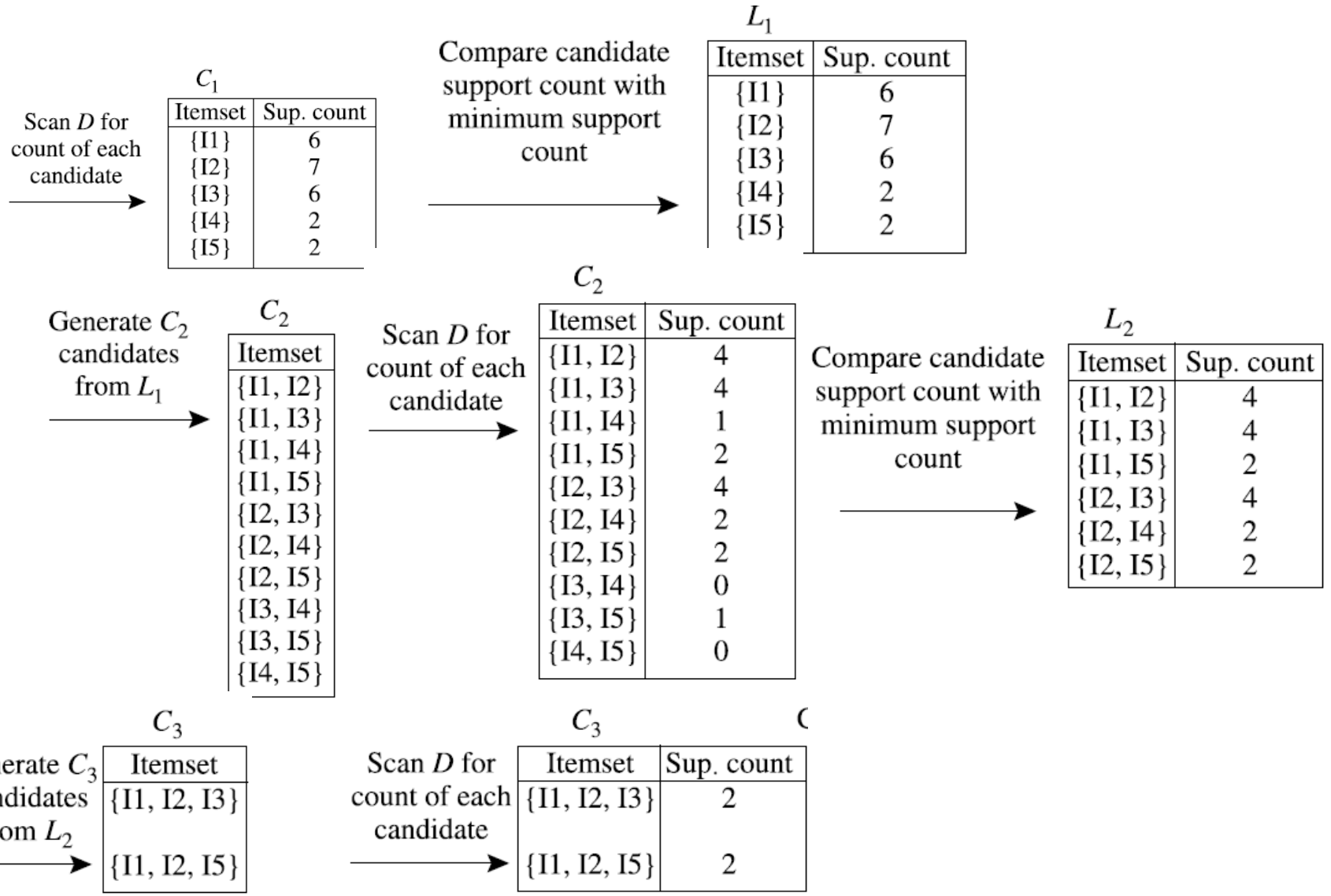
C_3

Itemset
{I1, I2, I3}
{I1, I2, I5}

The Apriori Algorithm—Another Example with Sup_{min} = 2

Transactional Data for an *AllElectronics* Branch

TID	*List of item_IDs*
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3



The Apriori Algorithm—Another Example with Sup_{min} = 2

Transactional Data for an *AllElectronics* Branch

TID	*List of item_IDs*
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

Scan D for count of each candidate →

C_1

Itemset	Sup. count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

Compare candidate support count with minimum support count →

L_1

Itemset	Sup. count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

Generate C_2 candidates from L_1 →

C_2

Itemset
{I1, I2}
{I1, I3}
{I1, I4}
{I1, I5}
{I2, I3}
{I2, I4}
{I2, I5}
{I3, I4}
{I3, I5}
{I4, I5}

Scan D for count of each candidate →

C_2

Itemset	Sup. count
{I1, I2}	4
{I1, I3}	4
{I1, I4}	1
{I1, I5}	2
{I2, I3}	4
{I2, I4}	2
{I2, I5}	2
{I3, I4}	0
{I3, I5}	1
{I4, I5}	0

Compare candidate support count with minimum support count →

L_2

Itemset	Sup. count
{I1, I2}	4
{I1, I3}	4
{I1, I5}	2
{I2, I3}	4
{I2, I4}	2
{I2, I5}	2

Generate C_3 candidates from L_2 →

C_3

Itemset
{I1, I2, I3}
{I1, I2, I5}

Scan D for count of each candidate →

C_3

Itemset	Sup. count
{I1, I2, I3}	2
{I1, I2, I5}	2

Compare candidate support count with minimum support count →

L_3

Itemset	Sup. count
{I1, I2, I3}	2
{I1, I2, I5}	2

Parallelize?

- Frequent 1 itemsets

- Frequent 1 itemsets

- generating candidate “k+1-itemset” (self-join and pruning)

- Frequent 1 itemsets
- generating candidate “k+1-itemset” (self-join and pruning)
- Generating frequent “k+1-itemsets” (consult DB)

Frequent 1 itemsets

Tid	Items bought
10	Beer, Nuts, Diaper
20	Beer, Coffee, Diaper
30	Beer, Diaper, Eggs
40	Nuts, Eggs, Milk
50	Nuts, Coffee, Diaper, Eggs, Milk

Generating candidate “k+1-itemset” (self-join and pruning)

L_3={*abc, abd, acd, ace, bcd*}

Generating frequent “k+1-itemsets” (consult DB)

- Frequent 1 itemsets
  - Word count with combine
- Self-join and pruning – generating “k+1-itemset” candidates
  - Map
    - Frequent K-itemsets are broadcasted
    - Input: <“self-join”, (partial k-itemsets)>
      - Self-join each k-itemset with broadcasted k-itemsets
    - Output: <“k+1-itemset”, 1>
  - Reduce
    - Frequent K-itemsets are broadcasted
    - Input: <“k+1-itemset, “list of 1’s”>
      - Prune based on broadcasted k-itemsets
    - Output: <“candidate k+1-itemset”, 1>
- Generating frequent “k+1-itemsets”
  - Map
    - Input: <“candidate”, “list of candidate k+1-itemsets”>
    - Read partial DB
    - Output: <“k+1-itemset”, “no of occurrences in partial DB”>
  - Reduce
    - Input: <“k+1-itemset”, “list of no of occurrences in partial DB”>
    - Output: <“k+1-itemset”, “addition of occurrences”>

Applications of Frequent Pattern Analysis

- Basket data analysis

Applications of Frequent Pattern Analysis

- Basket data analysis
- Web log (click stream) analysis

Questions to ask

Questions to ask

- Number of transactions?

Questions to ask

- Number of transactions?
- Minimum support?

Questions to ask

- Number of transactions?
- Minimum support?
- Number of total items?

Questions to ask

- Number of transactions?
- Minimum support?
- Number of total items?
- Number of frequent 1-itemsets?

Questions to ask

- Number of transactions?
- Minimum support?
- Number of total items?
- Number of frequent 1-itemsets?
- Number of frequent 2-itemsets?

Questions to ask

- Number of transactions?
- Minimum support?
- Number of total items?
- Number of frequent 1-itemsets?
- Number of frequent 2-itemsets?
- What's the longest frequent itemset?

Questions to ask

- Number of transactions?
- Minimum support?
- Number of total items?
- Number of frequent 1-itemsets?
- Number of frequent 2-itemsets?
- What's the longest frequent itemset?
- Number of total frequent itemsets?

Questions to ask

- Number of transactions?
- Minimum support?
- Number of total items?
- Number of frequent 1-itemsets?
- Number of frequent 2-itemsets?
- What's the longest frequent itemset?
- Number of total frequent itemsets?
- Number of total rules?

Questions to ask

- Number of transactions?
- Minimum support?
- Number of total items?
- Number of frequent 1-itemsets?
- Number of frequent 2-itemsets?
- What's the longest frequent itemset?
- Number of total frequent itemsets?
- Number of total rules?
- Confidence?

Questions to ask

- Number of transactions?
- Minimum support?
- Number of total items?
- Number of frequent 1-itemsets?
- Number of frequent 2-itemsets?
- What's the longest frequent itemset?
- Number of total frequent itemsets?
- Number of total rules?
- Confidence?
- How long does it take to compute all the frequent itemsets?

Classification

Big Data Analytics

Readings

- “Data Mining – Concepts and Techniques” by Jiawei Han, Micheline Kamber, Jian Pei, third edition
  - Chap. 8

Chapter 8. Classification: Basic Concepts

- Classification: Basic Concepts
- Decision Tree Induction
- Bayes Classification Methods
- Rule-Based Classification
- Model Evaluation and Selection
- Techniques to Improve Classification Accuracy: Ensemble Methods
- Summary

Supervised vs. Unsupervised Learning

- Supervised learning (classification)
  - Supervision: The training data (observations, measurements, etc.) are accompanied by **labels** indicating the class of the observations
  - New data is classified based on the training set
- Unsupervised learning (clustering)
  - The class labels of training data is unknown
  - Given a set of measurements, observations, etc. with the aim of establishing the existence of classes or clusters in the data

Prediction Problems: Classification vs. Numeric Prediction

- Classification
  - predicts categorical class labels (discrete or nominal)
  - classifies data (constructs a model) based on the training set and the values (class labels) in a classifying attribute and uses it in classifying new data
- Numeric Prediction
  - models continuous-valued functions, i.e., predicts unknown or missing values
- Typical applications
  - Credit/loan approval:
  - Medical diagnosis: if a tumor is cancerous or benign
  - Fraud detection: if a transaction is fraudulent
  - Web page categorization: which category it is